

Programdokumentáció: Pálcika Számlálás

Ez a szkript képes a megadott mappából (testing/) beolvasni a képeket, azokra a felhasználó által beállított különféle zajokat (Salt & Pepper, Gauss-zaj) keverni, majd képfeldolgozási algoritmusok (Canny éldetektálás, Morfológiai zárás, Hough-vonaltranszformáció) segítségével megkeresni és megszámolni a képen található egyenes szakaszokat („pálcikákat”).

1. Előfeltételek

(fejlesztőkörnyezetnek a PyCharm 2025.3.3-as verzióját használtam)

A kód futtatásához az alábbi Python könyvtárak szükségesek:

- opencv-python (4.13.0-ás verzió)
- numpy (2.4.4-es verzió)
- os
- math

A képeket a szkript mellett található testing/ nevű mappába kell helyezni .jpg, .jpeg vagy .png kiterjesztéssel.

2. Globális Változók és Inicializálás

- images: A beolvasott eredeti képeket (NumPy tömböket) tároló lista.
- imgPath: A beolvasott képek elérési útvonalait tároló lista.
- currentImgIndex: Az aktuálisan megjelenített kép indexe.
- windowName: A megjelenítő ablak neve ('Palcikak').
- noisyImgs: Eltárolja a már kiszámított zajos/feldolgozott képeket, hogy ne genrálja újra minden képkockában, ha nem változtak a csúszkák.
- tackbarValues: A csúszkák alapállása.

3. Funkciók és Működési Logika

A program indításkor végignézi a testing/ mappát. Ha nem talál érvényes képet, hibát dob, különben feltölti az images listát illetve kiírja a jelenleg beolvasott kép útvonalát.

Interaktív Csúszkák (Trackbars)

Három csúszka kerül létrehozásra a zajszintek vezérlésére alap opencv komponenssel (0-100-ig terjedő skálán, de a végső valószínűsége lejjebb lett osztva):

1. **'S' (Salt):** Só-zaj (fehér pixelek) valószínűsége. (/200)
2. **'P' (Pepper):** Bors-zaj (fekete pixelek) valószínűsége. (/200)
3. **'Gauss':** Gauss-féle fehérzaj intenzitása. (0,9-szerese)

A csúszkák elmozdulásakor a tackbarValueChange függvény fut le, ami ki is írja az aktuális értékeket a konzolra.

4. A Fő Programciklus (while True)

A hurok folyamatosan figyeli a felhasználói interakciókat és végzi a képfeldolgozást. Ha a felhasználó bezárja az ablakot (X gomb) vagy a megadott kilépő gombot megnyomja, a ciklus megszakad.

A feldolgozás lépései:

A) Zajgenerálás és Előszűrés

Ha a csúszkák értéke megváltozik, a noisyImgs ürül. Ha az aktuális kép még nincs feldolgozva az új értékekkel, a program létrehoz egy másolatot az eredeti képből, majd:

1. **Só-zaj:** Véletlenszerű maszk alapján fehér (255) pixeleket helyez el, majd egy 3×3-as Medián szűrővel (cv2.medianBlur) azonnal megpróbálja elsimítani.
2. **Bors-zaj:** Véletlenszerű maszk alapján fekete (0) pixeleket helyez el, amit szintén egy 3×3-as Medián szűrő követ.
3. **Gauss-zaj:** Normál eloszlású zajt ad a képhez, levágja a [0, 255] tartományon kívüli értékeket (np.clip (túlsordulások ellen kellett egy float konverzió)), majd egy 5×5-ös **Gauss-szűrővel** (cv2.GaussianBlur) mossa el.

B) Éldetektálás és Morfológia

- **Szürkeárnyalat:** A kép szürkeárnyalatossá alakul (cv2.cvtColor).
- **Canny éldetektáló:** Megkeresi a kép éleit a 30-as és 190-es hiszterézis küszöbértékek között.
- **Morfológiai zárás (Closing):** Egy 14×14-es négyzetes struktúrájú elemmel (kernel) morfológiai zárást végez az éleken. Ez segít összekötni a zaj miatt megszakadt vonaldarabkákat.

C) Vonalak Detektálása és szűrése

A program a **HoughLinesP** (Probabilisztikus Hough-vonaltranszformáció) segítségével kigyűjti a szakaszokat. Mivel a vastagabb élek miatt egy pálcikára több párhuzamos vonalat is találhat a rendszer, egy egyedi szűrést alkalmaz:

1. **Rendezés:** Hosszúság alapján csökkenő sorrendbe rakja a talált vonalakat (pitagoraszt használva, gyökvonás elengedhető).

```
cleanLines.sort(key=lambda l: (l[2] - l[0]) ** 2 + (l[3] - l[1]) ** 2, reverse=True)
```

2. **Távolság alapú szűrés:** A leghosszabb (domináns) vonaltól kiindulva kiszámolja a többi vonal végpontjainak távolságát a domináns egyenestől.

```
d1 = abs((y2 - y1) * cx1 - (x2 - x1) * cy1 + x2 * y1 - y2 * x1) / lineLength (kezdő pont)
d2 = abs((y2 - y1) * cx2 - (x2 - x1) * cy2 + x2 * y1 - y2 * x1) / lineLength (végpont)
```

3. Ha egy vizsgált vonal mindkét végpontja **30 pixelnél közelebb** van a domináns vonalhoz, akkor azt ismétlődőnek tekinti és kihagyja.

D) Megjelenítés

A megmaradt, megszűrt vonalakat (lineListFinal) a program **vörös színnel**, rákapcsolja a képre, majd a bal felső sarokba szintén vörös színnel kiírja a pálcikák végső számát (palcikák: X) illetve a konzolra is kiíratom.

5. Billentyűzet Vezérlés (Hotkeys)

A cv2.waitKey(10) függvény 10 ms-onként figyeli a billentyűleütéseket:

Billentyű	Funkció
D	Következő kép betöltése. Ha az utolsó képnél járunk, visszaugrik az elsőre és üríti a gyorsítótárat.
A	Előző kép betöltése. Ha az első képnél járunk, az utolsóra ugrik és üríti a gyorsítótárat.
q vagy Esc	Kilépés a ciklusból, ezt követi az összes ablak bezárása